ECEA125-2P_B1
MICROELECTRONICS 2

pH-Meter Verilog:
Final Project Documentation

Submitted by:
Baguinon, Sofia Maia A.
Eusebio, Alvin S.
Padilla, Kristian Emmanuel T.

Instructor:
Engr. Febus Reidj G. Cruz

System Specifications

Table 1. Measurement Specifications

Parameter	Specification	Notes / Basis
pH Measurement Range	*0.00 - 14.00 pH*	Standard pH scale
Resolution	*0.01 pH*	Required system precision
Display Format	*4-digit (e.g., 0700 = 7.00)*	Matches 0.01 resolution
Calibration Points	*3-point (pH 4, 7, 10)*	Buffer solutions
Measurement Type	*Linear (y = mx + b)*	Based on calibration model

Table 2. Electrical Specifications

Parameter	Specification	Notes / Basis
Supply Voltage	*3 V*	2×1.5V battery / button cells
Voltage Range (Sensor)	*0 - 3 V*	Assumed full-scale input
Sensitivity	*~0.21 V/pH*	3V / 14 pH
Resolution (Voltage)	*~2.1 mV per 0.01 pH*	Derived from sensitivity

Table 3. Data Conversion Specifications

Parameter	Specification	Notes / Basis
ADC Resolution	*12-bit*	Provides required precision
Input Signal Width	*12 bits (sense[11:0])*	From system design
Effective Resolution Needed	*≥10.48 bits*	To meet 0.01 pH
Input Type	*Digital (post-ADC)*	Assumed in implementation

System Overview

The system measures pH by processing sensor input through calibration, stability checking, computation, and display modules. Each module is verified individually, and the complete system is validated using the *ph_meter_tb* testbench.

Modules

1. ***Calibration Engine (calibration_engine.v)***

   Computes the linear calibration coefficients (slope and intercept) based on three reference sensor readings.

   Inputs:
   - y1_call[15:0], y2_call[15:0], y3_call[15:0] - calibration points
   - clk - system clock
   - reset - system reset
   - calib_store_en - enables coefficient storage

   Outputs:
   - slope_dbg[31:0] - computed slope
   - intercept_dbg[31:0] - computed intercept

2. ***Coefficient Storage (coeff_storage.v)***

   Stores the computed calibration coefficients and provides stable outputs for use in pH computation.

   Inputs:
   - clk - system clock
   - reset - system reset
   - calib_store_en - write enable signal
   - slope_dbg[31:0], intercept_dbg[31:0] - incoming calibration values

   Outputs:
   - slope_dbg[31:0] - stored slope
   - intercept_dbg[31:0] - stored intercept

3. *Stability Checker (stability_checker.v)*

Monitors incoming sensor values and determines when the signal is stable enough for processing.

Inputs:

- sense[11:0] - raw sensor input
- clk - system clock
- reset - system reset

Outputs:

- sense_latched_dbg[11:0] - stabilized sensor value
- ph_ready_dbg - indicates stable data is ready

4. *pH Compute (ph_compute.v)*

Calculates the pH value using the calibrated linear equation:

$$\mathbf{pH = slope \times sensor + intercept}$$

Inputs:

- sense_latched_dbg[11:0] - stabilized sensor value
- slope_dbg[31:0] - calibration slope
- intercept_dbg[31:0] - calibration intercept
- clk - system clock

Outputs:

- ph_value_dbg[31:0] - computed pH value

5. *Display Driver (display_driver.v)*

Converts the computed pH value into multiplexed signals for a 7-segment display.

Inputs:

- ph_value_dbg[31:0] - computed pH value
- clk - system clock
- reset - system reset

Outputs:

- an[3:0] - digit enable signals
- seg[6:0] - segment control signals
- dp - decimal point control

6. ***Main Module (main.v)***

Integrates all system modules, managing the flow from sensor input to final display output.

Inputs:

- sense_latched_dbg[11:0] - raw sensor input
- clk - system clock
- reset - system reset

Outputs:

- an[3:0], seg[6:0], dp - display outputs
- ph_ready_dbg - system ready indicator

7. ***Standby (standby.v)***

Controls system behavior during idle or inactive states, preventing unnecessary computation when no valid input is present.

Inputs:

- clk - system clock
- reset - system reset
- control signals (e.g., enable or idle conditions)

Outputs:

- system enable/disable signals

System Integration

1. ***Overall System Flow***

- The system operates as a sequential pipeline that converts sensor input into a displayed pH value.
- The sensor input (*sense*) is processed by the stability checker. When stable, the value is latched into *sense_latched_dbg* and *ph_ready_dbg* is asserted.
- Calibration coefficients (*slope_dbg, intercept_dbg*) are generated by the calibration engine upon *calib_store_en* and stored for use during operation.
- The pH compute module uses *sense_latched_dbg* together with *slope_dbg* and *intercept_dbg* to produce the computed output *ph_value_dbg*.
- The computed pH value is passed to the display driver, which generates multiplexed display signals (*seg[6:0], an[3:0], dp*) when *ph_ready_dbg = 1.*

Table 4. Control and Debug Signals

Signal	**Type**	**Function**	**Description**
clk	Input	System Clock	Drives all modules and synchronizes system operations
reset	Input	Initialization	Resets the system to its initial state and clears internal registers
calib_store_en	Input	Calibration Control	Triggers storage of calibration inputs and recomputation of slope and intercept
ph_ready_dbg	Output	Output Enable	Indicates that the input signal is stable and the computed pH value is valid
state_dbg[1:0]	Output	State Monitoring	Represents the current FSM state (e.g., standby, compute/display)
sense_latched_dbg[11:0]	Output	Latched Input	Stores the stabilized sensor value used for computation
ph_value_dbg[31:0]	Output	Computed Output	Represents the computed pH value in fixed-point format
slope_dbg[31:0]	Output	Calibration Parameter	Stores the computed slope from calibration
intercept_dbg[31:0]	Output	Calibration Parameter	Stores the computed intercept from calibration
seg[6:0]	Output	Display Control	Controls the segments of the 7-segment display
an[3:0]	Output	Digit Select	Enables specific digits for multiplexed display

Testbench Methodology

1. Verification Approach

The system was verified using a **two-level testbench strategy** consisting of:

- **Module-level verification** using dedicated testbenches
- **System-level verification** using the integrated testbench *ph_meter_tb.v*

Each module was first validated independently to ensure correct functionality before integrating all components into the complete pH meter system.

2. Module-Level Testbenches

Table 5. Testbenches to verify individual modules

Module	Testbench File	Purpose
Calibration Engine	*calibration_engine_tb.v*	Verifies slope and intercept computation
Stability Checker	*stability_checker_tb.v*	Validates stable signal detection
pH Compute	*ph_compute_tb.v*	Checks correctness of pH calculation
Display Driver	*display_driver_tb.v*	Verifies 7-segment output behavior
Standby FSM	*standby_tb.v*	Confirms correct state transitions

2.1. *Stable Input Generation*

```
case (i % 3)
    0: sense = base;
    1: sense = base + 12'd1;
    2: sense = base + 12'd2;
endcase
```

Figure 1.

Small variations within ±2 LSB are applied to emulate real sensor noise. The system should treat these values as stable and allow computation.

2.2. *Noisy Input Rejection*

```
sense = base + 12'd6; #10;
sense = base - 12'd7; #10;
sense = base + 12'd5; #10;
sense = base - 12'd6; #10;
```

Figure 2.

Larger variations outside the tolerance window are applied. The system is expected to reject these inputs and prevent *ph_ready_dbg* from being asserted.

2.3. *Calibration / Recalibration Input*

```
cal_y1 = 16'sd1200;
cal_y2 = 16'sd2100;
cal_y3 = 16'sd3000;
calib_store_en = 1'b1;
```

Figure 3.

Known calibration points are provided to generate new slope and intercept values, verifying correct coefficient computation and storage.

3. *System-Level Verification (ph_meter_tb.v)*

- After module validation, the complete system was tested using:

tests/ph_meter_tb.v

- This testbench verifies the entire pipeline:

Sensor → Stability → Calibration → Computation → Display

3.1. *Measurement Execution*

```
stable_input(adc_value);
wait (ph_ready_dbg == 1'b1);
```

Figure 4.

A stable input is applied, and the testbench waits until the system asserts *ph_ready_dbg*, indicating that the computed pH value is valid.

4. *Verification Flow*

- Testbench stimulus is applied to the system modules.
- Simulation is executed using Icarus Verilog (*iverilog* + *vvp*).
- Terminal output is checked to verify correct execution and debug messages.
- The generated waveform file (*.vcd*) is analyzed using GTKWave.
- System behavior is validated by observing signal transitions and outputs.

Final Verification (GTKWave Results)

1. *Calibration Phase*

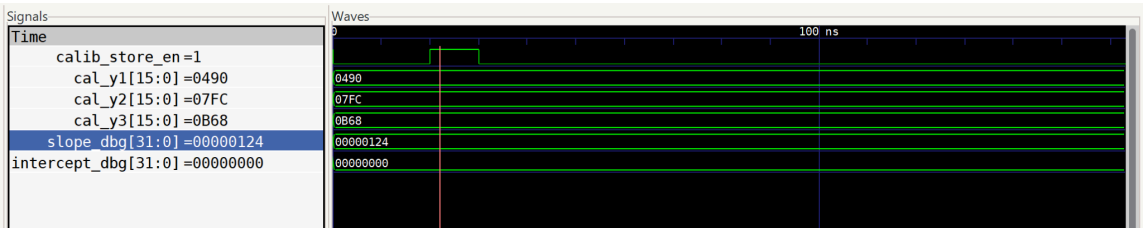

Figure 5. Calibration coefficient generation

At the moment calib_store_en is asserted, the system computes and stores the calibration coefficients using the provided reference points. The resulting slope (00000124 = 292) and intercept (0) confirm correct implementation of the calibration engine.

2. *Stability Detection*

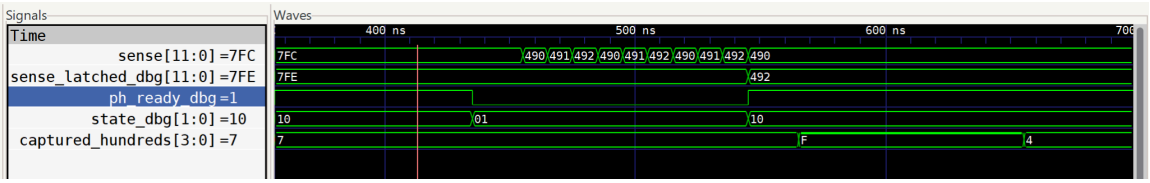

Figure 6. Stability detection and input latching

The input signal initially fluctuates due to noise. Once the signal stabilizes, the system asserts *ph_ready_dbg* and transitions from standby to display mode. The stable value is then latched into *sense_latched_dbg*, ensuring that only valid measurements are processed.

3. *pH Computation (Example: pH 8.36)*

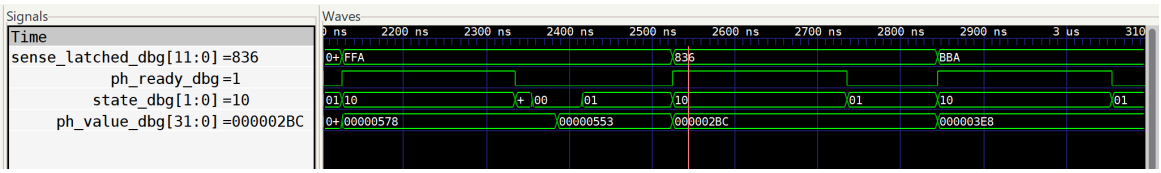

Figure 7. pH computation result (8.36)

After detecting a stable input, the system transitions to display mode and computes the corresponding pH value. The signal *ph_ready_dbg = 1* indicates a valid measurement, while *state_dbg = 10* confirms the display state. The computed output *ph_value_dbg = 836* corresponds to a pH value of 8.36.

4. *Resolution Check*

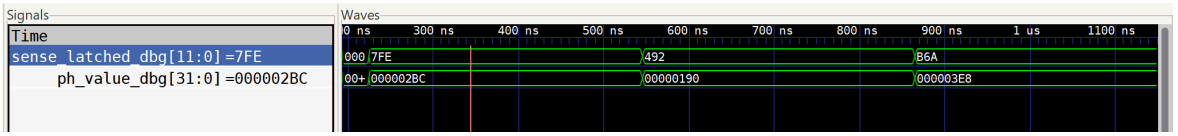

Figure 8. Resolution verification of pH computation

Small variations in the input ADC values result in corresponding changes in the computed pH output. This demonstrates that the system maintains correct resolution and accurately reflects input variations in the final pH measurement.

5. *Display Output*

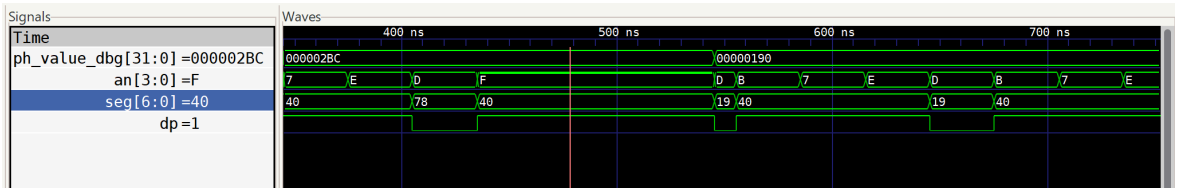

Figure 9. 7-segment display output

The computed pH value is translated into multiplexed 7-segment display signals. The an signal cycles through each digit position, while seg updates accordingly to represent the correct numerical value. The presence of dp confirms proper decimal point handling, demonstrating correct display operation.

6. *Low-end and High-end Range Validation (0 - 14 pH)*

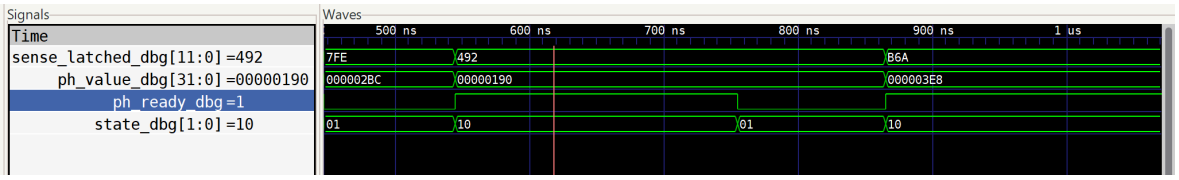

Figure 10. Low-range pH validation

A stabilized sensor input (*sense_latched_dbg = 492*) results in a computed pH value of 400 (≈ 4.00) *when ph_ready_dbg* is asserted. The system transitions to the valid computation/display state (*state_dbg = 10*), confirming correct operation within the lower pH measurement range.

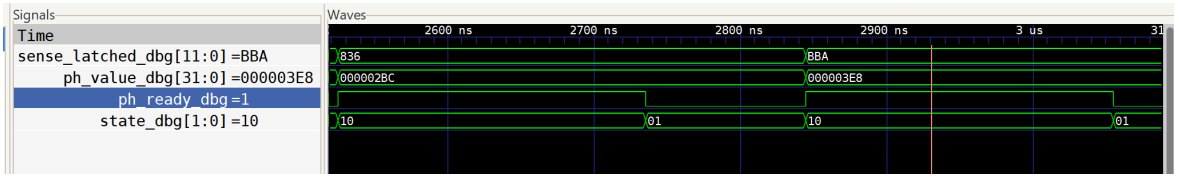

Figure 11. High-range pH validation

A high sensor input (*sense_latched_dbg = BBA*) results in a computed pH value of 1000 (≈10.00). The system asserts ph_ready_dbg and transitions to the valid output state (*state_dbg = 10*), confirming correct operation at the upper measurement range.

7. *Display Disable When System is Off*

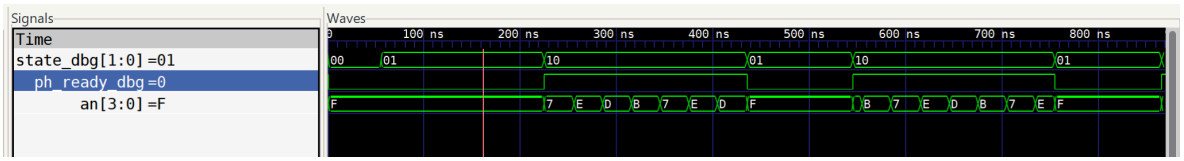

Figure 12. Display disabled during inactive state

When the system is not ready (*ph_ready_dbg = 0*), the FSM remains outside the computation state (*state_dbg = 01*), and all display digits are disabled (*an = 1111*). This prevents invalid or unstable values from being shown on the 7-segment display.

8. *Recalibration Behavior*

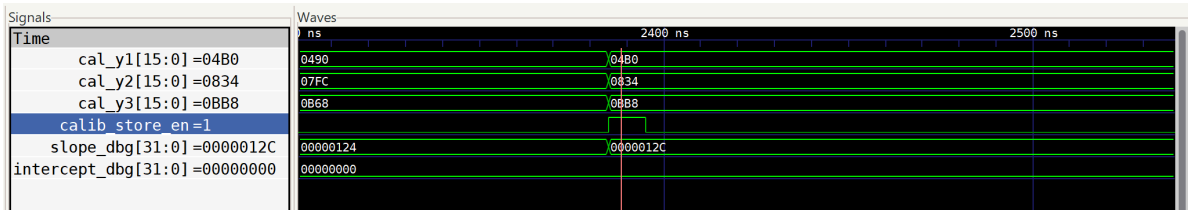

Figure 13. Recalibration coefficient update

New calibration inputs (*cal_y1, cal_y2, cal_y3*) are applied, followed by assertion of *calib_store_en.* Upon this event, the calibration engine recomputes the coefficients, as shown by the update in slope_dbg from 00000124 to 0000012C. This confirms that the system correctly performs recalibration and updates its internal parameters.

Division of Tasks

Housekeeping - Padilla

Source code:
Baguinon

1. main module
2. standby module
3. checkStability module
4. display module

Eusebio

5. calibration module
6. store module
7. compute module

Testbenches - Padilla

Documentation - Baguinon

Github Repository:
https://github.com/ketpadilla/ph-meter-verilog